Spring 2026

CS 5757: Optimization Methods for Robotics
Homework 3: Optimal Control

Due: Thursday, March 26, 2026, at 11:59pm on Gradescope, Total Points: 50

Learning goals for this problem set:

- **Problem 1:** Study how LQR and iLQR perform under noise, and understand the benefits of replanning.
- **Problem 2:** Explore using Sequential Convex Programming (SCP) to impose collision avoidance constraints.
- **Problem 3:** Read ALTRO; compare and contrast using AL methods to impose constraints with SQP.
- **Problem 4:** Use SCP-based MPC to track a trajectory under model mismatch; understand the role of trust regions, replanning horizon, and constraint handling.

Submission Instructions: Submit your writeup as a PDF to the *Written Assignment* on Gradescope. For the *Programming Assignment*, submit *only* your completed `solutions.py`.
You are **strongly encouraged** to typeset your homework submissions using LaTeX(if you do not know LaTeXyet, now is as good a time as any to learn it – you will need it in grad school anyway). If you opt for handwritten submissions, make sure that they are legible – if we cannot read it, we cannot grade it!

3.1 (i)LQR for Stochastic Systems

Recall the finite-horizon LQR problem with quadratic costs and linear dynamics:

$$\begin{aligned} \min_{\boldsymbol{u}_{0:T-1}} \quad & \frac{1}{2}\boldsymbol{x}_T^\top \boldsymbol{Q}_T \boldsymbol{x}_T + \sum_{t=0}^{T-1}\left(\frac{1}{2}\boldsymbol{x}_t^\top \boldsymbol{Q}_t \boldsymbol{x}_t + \frac{1}{2}\boldsymbol{u}_t^\top \boldsymbol{R}_t \boldsymbol{u}_t\right) \\ \text{s.t.} \quad & \boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u}_t. \end{aligned}$$

Now suppose the dynamics are subject to additive stochastic noise,

$$\boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u}_t + \boldsymbol{w}_t, \qquad \boldsymbol{w}_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}),$$

and the objective becomes minimizing the *expected* cumulative cost.

(a) ✎ (5 pts) Using dynamic programming, derive the optimal policy $\boldsymbol{u}_t^*(\boldsymbol{x}_t)$ by working backward from the terminal cost. Show that the value function takes the form

$$V_t(\boldsymbol{x}) = \frac{1}{2}\,\boldsymbol{x}^\top \boldsymbol{P}_t\,\boldsymbol{x} + d_t,$$

where $\boldsymbol{P}_t$ satisfies the same Riccati recursion as the deterministic case,

$$\boldsymbol{P}_t = \boldsymbol{Q}_t + \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\boldsymbol{A} - \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\boldsymbol{B}\big(\boldsymbol{R}_t + \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{B}\big)^{-1}\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{A},$$

and the optimal control $\boldsymbol{u}_t^* = -\boldsymbol{K}_t\,\boldsymbol{x}_t$ with $\boldsymbol{K}_t = (\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{B} + \boldsymbol{R}_t)^{-1}\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{A}$ is identical to the deterministic case. Derive an expression for d_t in terms of $\boldsymbol{P}_t$ and the noise covariance $\boldsymbol{\Sigma}$, and explain why the noise does not affect the optimal policy.

Hint: Apply the dynamic programming recursion from lecture, taking the expectation over $\boldsymbol{w}_t$ and using $\mathbb{E}[\boldsymbol{w}_t] = \boldsymbol{0}$.

For the remaining parts, we apply iLQR to the **acrobot**—an inverted double pendulum with a single actuator at the elbow.

(b) (3 pts) Given nonlinear dynamics $\boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t)$ and running cost $c(\boldsymbol{x}_t, \boldsymbol{u}_t)$, iLQR linearizes the dynamics and quadraticizes the cost around a nominal trajectory $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$:

$$\begin{aligned}
\boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t) &\approx \boldsymbol{f}(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) + \boldsymbol{A}_t\,\delta\boldsymbol{x}_t + \boldsymbol{B}_t\,\delta\boldsymbol{u}_t, \\
c(\boldsymbol{x}_t, \boldsymbol{u}_t) &\approx c(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) + \boldsymbol{q}_t^\top \delta\boldsymbol{x}_t + \boldsymbol{r}_t^\top \delta\boldsymbol{u}_t + \tfrac{1}{2}\delta\boldsymbol{x}_t^\top \boldsymbol{Q}_t\,\delta\boldsymbol{x}_t + \tfrac{1}{2}\delta\boldsymbol{u}_t^\top \boldsymbol{R}_t\,\delta\boldsymbol{u}_t + \delta\boldsymbol{x}_t^\top \boldsymbol{S}_t\,\delta\boldsymbol{u}_t,
\end{aligned}$$

where $\delta\boldsymbol{x}_t = \boldsymbol{x}_t - \bar{\boldsymbol{x}}_t$, $\delta\boldsymbol{u}_t = \boldsymbol{u}_t - \bar{\boldsymbol{u}}_t$, and $\boldsymbol{A}_t, \boldsymbol{B}_t$ are dynamics Jacobians and $\boldsymbol{Q}_t, \boldsymbol{R}_t, \boldsymbol{S}_t, \boldsymbol{q}_t, \boldsymbol{r}_t$ are cost Hessians and gradients evaluated at $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$. Implement `linearize(f, cost, s, a)` using JAX autodiff, returning $(\boldsymbol{Q}_t, \boldsymbol{R}_t, \boldsymbol{S}_t, \boldsymbol{q}_t, \boldsymbol{r}_t, \boldsymbol{A}_t, \boldsymbol{B}_t)$ where $\boldsymbol{S}_t = \frac{\partial^2 c}{\partial \boldsymbol{x}\,\partial \boldsymbol{u}}$.

(c) (3 pts) Implement `ilqr_backward_step(P, p, Q, R, S, q, r, A, B, reg)` $\to (\boldsymbol{P}_t, \boldsymbol{p}_t, \boldsymbol{K}, \boldsymbol{k})$. Given the value function $V_{t+1}(\delta\boldsymbol{x}) \approx \frac{1}{2}\delta\boldsymbol{x}^\top \boldsymbol{P}\,\delta\boldsymbol{x} + \boldsymbol{p}^\top \delta\boldsymbol{x}$ and the expansions from part (c), compute the action-value expansions $\boldsymbol{H}_{xx}, \boldsymbol{H}_{xu}, \boldsymbol{H}_{uu}, \boldsymbol{h}_x, \boldsymbol{h}_u$, solve for the optimal feedback gains $(\boldsymbol{K}, \boldsymbol{k})$, and propagate the value function backward to $(\boldsymbol{P}_t, \boldsymbol{p}_t)$.

Hint: Apply Levenberg–Marquardt regularization $\boldsymbol{H}_{uu} \leftarrow \boldsymbol{H}_{uu} + \texttt{reg}\cdot\boldsymbol{I}$ and use `jnp.linalg.solve` rather than an explicit inverse.

(d) (2 pts) Implement `simulate` in `solution.py`, which rolls out the system under additive Gaussian noise. Complete the two indicated lines to apply either the open-loop or closed-loop iLQR control at each timestep, depending on the bool `use_feedback`.

(e) (1 pts) Run `acrobot_ilqr.py`, which simulates the acrobot under both open-loop and closed-loop iLQR across increasing noise levels. Include the generated plot in your writeup.

(i) At low noise, how does closed-loop performance compare to open-loop? Why should we expect this from part (a)?

(ii) As noise increases, the closed-loop policy eventually degrades sharply. What assumption underlying the iLQR feedback gains is violated when the noise is large?

Solution:

3.2 Sequential Convex Programming

Consider a drone navigating through n_{obs} spherical obstacles, each with center $\boldsymbol{o}_i \in \mathbb{R}^3$ and radius r_i. The drone has radius r_d and state $\boldsymbol{x} = [\boldsymbol{p}^\top, \boldsymbol{v}^\top]^\top \in \mathbb{R}^6$. We want to find a collision-free trajectory minimizing

$$\min_{\boldsymbol{u}_{0:T-1}} \quad \frac{1}{2}(\boldsymbol{x}_T - \boldsymbol{x}_g)^\top \boldsymbol{P}\,(\boldsymbol{x}_T - \boldsymbol{x}_g) + \sum_{t=0}^{T-1} \left[\frac{1}{2}(\boldsymbol{x}_t - \boldsymbol{x}_g)^\top \boldsymbol{Q}\,(\boldsymbol{x}_t - \boldsymbol{x}_g) + \frac{1}{2}\boldsymbol{u}_t^\top \boldsymbol{R}\,\boldsymbol{u}_t\right] \tag{1}$$

$$\text{s.t.} \quad \boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t), \tag{2}$$

$$d_i(\boldsymbol{x}_t) \geq 0, \quad i = 1, \ldots, n_{\text{obs}}, \quad t = 0, \ldots, T, \tag{3}$$

which is nonconvex due to the nonlinear dynamics (2) and obstacle constraints (3). SCP solves this by iteratively linearizing both around the current nominal trajectory and solving the resulting convex QP, whose solution becomes the new nominal trajectory.

Parts (e) and (f) use CVXPY; see here for basic usage. Note that we will be using `cvx.Parameter`, which allows you to update problem data between solves without reconstructing the problem.

(a) (2 pts) Implement `signed_distance(state)`, where `state` has shape `(nx,)`. Return $\boldsymbol{d} \in \mathbb{R}^{n_{\text{obs}}}$, $d_i = \|\boldsymbol{p} - \boldsymbol{o}_i\|_2 - r_i - r_d$, using global constants `OBSTACLE_CENTERS (n_obs, 3)`, `OBSTACLE_RADII (n_obs,)`, and `DRONE_RADIUS`.

(b) (3 pts) The signed distance constraint $d_i(\boldsymbol{x}) = \|\boldsymbol{p} - \boldsymbol{o}_i\|_2 - r_i - r_d$ is nonconvex (the inequality is backwards), so we linearize the obstacle constraint $d_i(\boldsymbol{x}) \geq 0$ about a nominal state $\bar{\boldsymbol{x}}$:

$$\underbrace{\nabla_{\boldsymbol{x}} d_i(\bar{\boldsymbol{x}})^\top}_{\boldsymbol{g}_i}\, \boldsymbol{x} \;+\; \underbrace{d_i(\bar{\boldsymbol{x}}) - \nabla_{\boldsymbol{x}} d_i(\bar{\boldsymbol{x}})^\top \bar{\boldsymbol{x}}}_{h_i} \;\geq\; 0.$$

Show that any $\boldsymbol{x}$ satisfying this linearized constraint is *guaranteed* to satisfy $d_i(\boldsymbol{x}) \geq 0$. Does this hold for the signed-distance of a non-convex shape?

Hint: Prove that d_i is convex using the definition from class.

(c) (1 pts) Linearize $\boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t)$ around $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$ and show that the result can be written as $\boldsymbol{x}_{t+1} \approx \boldsymbol{A}_t \boldsymbol{x}_t + \boldsymbol{B}_t \boldsymbol{u}_t + \boldsymbol{c}_t$. Derive $\boldsymbol{A}_t$, $\boldsymbol{B}_t$, and $\boldsymbol{c}_t$.

(d) (2 pts) Implement `affinize_dynamics(f, s_bar, u_bar)` using JAX autodiff, returning $(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{c})$ as derived in part (c). Inputs `s_bar (nx,)` and `u_bar (nu,)`; outputs have shapes `(nx, nx)`, `(nx, nu)`, `(nx,)`.

You should also implement `affinize_constraint(g, s_bar)`, returning $(\boldsymbol{G}, h)$ where $\boldsymbol{G} = \frac{\partial \boldsymbol{d}}{\partial \boldsymbol{x}_t}$ and $h = g(\bar{\boldsymbol{x}}) - \boldsymbol{G}\bar{\boldsymbol{x}}$, following the same pattern as `affinize_dynamics`. Input `s_bar (nx,)`; outputs have shapes `(n_obs, nx)` and `(n_obs,)`.

(e) (2 pts) Complete `build_drone_scp`. The cost, dynamics constraints, and initial condition are already provided. Add the linearized obstacle constraints for all $t \in \{1, \ldots, T\}$:

$$\boldsymbol{G}_t\, \boldsymbol{x}_t + \boldsymbol{h}_t \geq 0,$$

using parameters `G_p (T+1, n_obs, nx)` and `h_p (T+1, n_obs)`.

(f) (3 pts) Implement `solve_scp`. At each iteration, use `affinize_dynamics` and `affinize_constraint` to compute $(\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{c}_t)$ and $(\boldsymbol{G}_t, h_t)$ along the current nominal trajectory `s_nom (T+1, nx)`, `u_nom (T, nu)`, assign them to the CVXPY parameters `(A_p, B_p, c_p) (T, nx, nx)`, `(T, nx, nu)`, `(T, nx)` and `(G_p, h_p) (T+1, n_obs, nx)`, `(T+1, n_obs)`, and solve. Run `problem2.py` and include the 3D trajectory plot in your writeup.

(g) (1 pts) In part (b) you showed that the linearized obstacle constraints are *conservative*: satisfying them guarantees collision avoidance at every SCP iteration. Does the same hold for the linearized dynamics? What does this asymmetry imply about the dynamic feasibility of an unconverged solution?

Solution:

3.3 ALTRO

For this question, we will read *ALTRO: A Fast Solver for Constrained Trajectory Optimization* by Taylor Howell, Brian Jackson, and Zachary Manchester. The paper extends iLQR to handle constraints via an augmented Lagrangian method.

(a) ✎ (1 pts) Why can't iLQR handle state or input constraints (e.g., joint limits, obstacle avoidance)?

(b) ✎ (2 pts) Does ALTRO guarantee constraint satisfaction at each iLQR iteration, or only upon convergence of the augmented Lagrangian outer loop? Are the trajectories estimated by ALTRO dynamically feasible at each iteration? How does this compare to the SQP approach you implemented in the previous problem?

(c) ✎ (1 pts) What is the role of the regularization parameter ρ in the δu update given by Equation (16)? Why is this needed in practice for general trajectory optimization problems?

(d) ✎ (2 pts) The authors propose an "infeasible start" method in Section III.B. Give a concrete robotics scenario where this would be practically useful, and explain why a standard (dynamically feasible) initialization would struggle in that setting.

(e) ✎ (2 pts) Suppose our dynamics involve intermittent contact (e.g., a humanoid walking across the ground), so that the dynamics gradients $\partial f/\partial x$, $\partial f/\partial u$ are unreliable or discontinuous. How would you expect ALTRO's iLQR backward pass to behave? Be specific about which computational step breaks down.

(f) ✎ (2 pts) What is the fundamental limitation of enforcing constraints via a pure penalty method (adding $\rho\|c(x,u)\|^2$ to the cost) inside iLQR, and how does the Augmented Lagrangian formulation resolve it? Your answer should address both constraint satisfaction and numerical conditioning of the Riccati backward pass.

Solution:

3.4 Model Predictive Control

Even a well-optimized trajectory can fail in practice when the planning model differs from the real system. Here, we plan using a simplified RK4 model of the acrobot but execute on a full MuJoCo simulator with joint damping and perturbed inertial parameters. Under open-loop execution, small model errors accumulate over the horizon and the trajectory degrades. Model Predictive Control (MPC) addresses this by replanning at each step: we solve an SCP subproblem from the current observed state, execute only the first few controls, then replan from the new state. This closed-loop structure lets the planner continually correct for model mismatch rather than committing to a single fixed plan.

Because linearization is only accurate near the nominal trajectory, each SCP subproblem restricts the solution to a *trust region* — a neighborhood of the current nominal trajectory within which the linearized model is a reliable approximation. Parts (a) and (b) analyze the ℓ_2 trust region subproblem theoretically, since it is easier to reason about. For the MPC implementation, we use an ℓ_∞ trust region $\|\boldsymbol{d}\|_\infty \leq \rho$, which reduces to box constraints and can be solved as a standard QP without second-order cone programming.

(a) (2 pts) Ignoring the dynamics and other constraints, the SCP subproblem at each iteration is a quadratic program with a trust region constraint:

$$\begin{aligned} \min_{\boldsymbol{d}} \quad & \frac{1}{2}\boldsymbol{d}^\top \boldsymbol{H}\, \boldsymbol{d} + \boldsymbol{g}^\top \boldsymbol{d} \\ \text{s.t.} \quad & \|\boldsymbol{d}\|_2^2 \leq \Delta^2, \end{aligned}$$

where $\boldsymbol{H}$ is the (possibly indefinite) Hessian of the linearized cost, $\boldsymbol{g}$ is the cost gradient, and $\Delta > 0$ is the trust region radius. Write the KKT conditions for this problem and show that the optimal $\boldsymbol{d}^*$ satisfies, for some $\mu^* \geq 0$:

$$(\boldsymbol{H} + \mu^* \boldsymbol{I})\, \boldsymbol{d}^* = -\boldsymbol{g}.$$

(b) (2 pts) Show that if $\boldsymbol{H}$ has a negative eigenvalue, the unconstrained quadratic program

$$\min_{\boldsymbol{d}} \quad \frac{1}{2}\boldsymbol{d}^\top \boldsymbol{H}\, \boldsymbol{d} + \boldsymbol{g}^\top \boldsymbol{d}$$

is unbounded below. What does this imply about solving SCP subproblems without a trust region when the Hessian is indefinite?

Remark: It is easy to show that when $\boldsymbol{H}$ is indefinite, the trust region constraint must be active ($\mu^* > 0$). From part (a), we can see this means that μ^* is implicitly acting as regularization.

(c) (2 pts) Complete `build_acrobot_scp` in `solution.py`, which constructs a CVXPY problem for one SCP subproblem. The cost function and affine dynamics constraints are already provided. The decision variables are `s`, shape `(T+1, nx)`, and `u`, shape `(T, nu)`, and the nominal trajectory is given by the parameters `s_bar_p`, shape `(T+1, nx)`, and `u_bar_p`, shape `(T, nu)`. Add the following constraints:

- **Control bounds** for all $t \in \{0, \ldots, T-1\}$:

$$-u_{\max} \;\leq\; u_t \;\leq\; u_{\max}.$$

- ℓ_∞ **trust regions** for all $t \in \{0, \ldots, T-1\}$:

$$\|u_t - \bar{u}_t\|_\infty \le \rho, \qquad \|s_t - \bar{s}_t\|_\infty \le \rho \quad (t > 0),$$

  where the state trust region skips $t = 0$ since s_0 is already fixed by the initial condition. The trust region radius ρ and bound $u_{\max}$ are given by `trust_region` and `u_bound`, respectively.

(d) ✎ (1 pts) Run `problem3.py`. Include the generated plot in your writeup. The open-loop SCP plan fails on MuJoCo while receding-horizon MPC succeeds. Why does replanning at each step outperform using the fixed iLQR feedback gains?

(e) ✎ (1 pts) Reduce the MPC horizon to $T = 5$ and rerun. Include the resulting plot in your writeup. The acrobot fails to swing up. Why does MPC suddenly fail with a short horizon?

(f) ✎ (2 pts) Propose two modifications to the MPC formulation that could resolve the short-horizon problem. For each, describe the tradeoff it introduces.

Solution:

3.5 AI Usage Survey

✎ (2 pts) Please describe how you used AI (if at all) while completing this homework. Note that you will not be penalized for AI usage, regardless of how extensively you used it, as long as you acknowledge it. We are generally curious about how we can improve the course material and better help with your learning!

References